

Resenha do Artigo Design by Contract

Artur Bomtempo Colen

Engenharia de Software

Pontifícia Universidade Católica de Minas Gerais (PUC Minas)

Belo Horizonte, Brasil

abcolen@sga.pucminas.br

Resumo—Este trabalho apresenta uma análise do capítulo *Design by Contract*, de Bertrand Meyer. O texto aborda uma proposta de desenvolvimento de software baseada na ideia de contratos entre componentes de um sistema, definindo responsabilidades, obrigações e garantias entre clientes e fornecedores de serviços. O autor apresenta conceitos como pré-condições, pós-condições, invariantes e tratamento de exceções, demonstrando como essas técnicas podem contribuir para o desenvolvimento de sistemas mais corretos, robustos e confiáveis. Esta resenha tem como objetivo discutir os principais conceitos apresentados no artigo, demonstrar o entendimento geral sobre a abordagem Design by Contract e analisar sua aplicação prática no desenvolvimento de software moderno.

Index Terms—engenharia de software, Design by Contract, confiabilidade, orientação a objetos, qualidade de software

I. INTRODUÇÃO

A confiabilidade de software é um dos principais desafios da engenharia de software. À medida que os sistemas se tornam mais complexos, distribuídos e reutilizáveis, cresce também a necessidade de garantir que os componentes funcionem corretamente em diferentes contextos.

No artigo, Bertrand Meyer apresenta a abordagem *Design by Contract* (DbC), baseada na ideia de que módulos de software devem estabelecer contratos explícitos entre si. Esses contratos especificam claramente quais condições devem ser satisfeitas antes da execução de uma operação e quais resultados devem ser garantidos após sua execução. [oai_citation : 0original – paper.pdf](sediment : //file000000094d4720eb93bc17500790f29)

O autor argumenta que muitos problemas relacionados à confiabilidade surgem justamente da falta de definições claras sobre responsabilidades entre diferentes partes de um sistema.

Além disso, o artigo discute como essa abordagem se integra ao paradigma orientado a objetos e como pode contribuir para melhorar aspectos como reutilização, manutenção e robustez do software.

II. A IDEIA DE CONTRATO EM SOFTWARE

O conceito central apresentado no artigo é a analogia entre contratos do mundo real e contratos de software.

Assim como em contratos jurídicos existem obrigações e benefícios para ambas as partes, no software também deve existir uma definição clara das responsabilidades entre quem utiliza um serviço e quem o fornece.

Segundo Meyer, um contrato define:

- o que o cliente deve garantir;

- o que o fornecedor deve executar;
- quais resultados devem ser obtidos;
- quais limitações existem para ambas as partes.

Essa ideia é utilizada para estruturar rotinas e componentes de maneira mais organizada e previsível.

O artigo enfatiza que contratos explícitos ajudam a reduzir ambiguidades, melhorando a comunicação entre desenvolvedores e tornando os sistemas mais fáceis de compreender.

Outro ponto importante é que os contratos servem não apenas como documentação, mas também como mecanismo de validação do comportamento do sistema durante sua execução.

III. PRÉ-CONDIÇÕES, PÓS-CONDIÇÕES E INVARIANTES

O artigo apresenta três conceitos fundamentais da abordagem Design by Contract: pré-condições, pós-condições e invariantes.

As pré-condições representam requisitos que devem ser satisfeitos antes da execução de uma operação. Caso essas condições não sejam atendidas, a responsabilidade do erro pertence ao cliente que realizou a chamada da operação.

Já as pós-condições representam garantias fornecidas pela rotina após sua execução. Elas descrevem o estado esperado do sistema depois que a operação termina corretamente.

Os invariantes, por sua vez, representam propriedades que devem permanecer verdadeiras durante todo o ciclo de vida de um objeto. Eles garantem a consistência global da classe e ajudam a manter o sistema em estados válidos. [oai_citation : 1original – paper.pdf](sediment : //file000000094d4720eb93bc17500790f29)

O autor demonstra esses conceitos utilizando exemplos na linguagem Eiffel, criada para suportar diretamente os mecanismos de contrato.

Um dos pontos mais interessantes é que os contratos permitem especificar o comportamento esperado do software de maneira formal, mas ainda relativamente compreensível para desenvolvedores.

Além disso, a presença desses mecanismos facilita a identificação de erros, já que violações de contrato indicam precisamente qual responsabilidade não foi cumprida.

IV. CONFIABILIDADE E QUALIDADE DE SOFTWARE

Outro tema importante discutido no artigo é a relação entre contratos e confiabilidade de software.

Segundo Meyer, muitas abordagens tradicionais utilizam programação defensiva, onde cada módulo tenta verificar continuamente possíveis erros e situações inesperadas.

O autor critica essa abordagem, argumentando que ela aumenta a complexidade do sistema e gera redundância desnecessária.

Na abordagem baseada em contratos, cada responsabilidade é claramente atribuída a uma das partes. Isso reduz verificações redundantes e simplifica a arquitetura do sistema.

Além disso, contratos ajudam a melhorar a documentação do software, pois descrevem formalmente o comportamento esperado das rotinas.

Outro benefício importante é a possibilidade de utilizar contratos como mecanismo de depuração e testes. Durante a execução do sistema, violações de pré-condições, pós-condições ou invariantes podem indicar falhas de implementação ou uso incorreto das operações.

O artigo também destaca que contratos contribuem significativamente para reutilização de software, já que componentes reutilizáveis precisam possuir comportamentos bem definidos e previsíveis.

V. TRATAMENTO DE EXCEÇÕES

O texto também discute o tratamento de exceções e situações anormais.

Segundo o autor, exceções devem ser utilizadas apenas em casos realmente excepcionais, e não como substituição para controle de fluxo comum.

A abordagem Design by Contract ajuda a definir de maneira clara quais condições devem ser verificadas pelo cliente e quais devem ser tratadas pelo fornecedor.

Isso reduz ambiguidades sobre responsabilidades no tratamento de erros.

O artigo também apresenta críticas ao uso excessivo de programação defensiva e mecanismos tradicionais de exceções, argumentando que muitos sistemas se tornam excessivamente complexos devido ao acúmulo de verificações redundantes.

Outro ponto importante é que contratos tornam mais fácil identificar a origem de falhas, já que uma violação indica exatamente qual parte do acordo foi descumprida.

VI. ENTENDIMENTO GERAL DO ARTIGO

A partir da leitura do artigo, é possível perceber que Design by Contract não é apenas uma técnica de programação, mas uma filosofia de desenvolvimento de software baseada em responsabilidade explícita.

O principal entendimento obtido é que sistemas confiáveis dependem não apenas de boas implementações, mas também de especificações claras e bem definidas.

Outro ponto importante é que a abordagem promove uma separação mais organizada de responsabilidades entre módulos, evitando ambiguidades e reduzindo a complexidade geral do sistema.

Além disso, ficou claro que contratos ajudam significativamente na manutenção e evolução do software, pois facilitam

o entendimento sobre o comportamento esperado de cada componente.

O artigo também evidencia como a orientação a objetos se beneficia dessa abordagem, principalmente em cenários de reutilização de componentes e herança.

Mesmo sendo uma proposta criada há muitos anos, diversos conceitos apresentados continuam extremamente relevantes para o desenvolvimento moderno.

VII. APLICAÇÃO PRÁTICA

Na prática, os conceitos de Design by Contract podem ser aplicados em diferentes tipos de sistemas.

Um exemplo é o desenvolvimento de APIs e serviços web. Ao definir claramente os dados esperados em requisições e os resultados garantidos nas respostas, é possível reduzir erros de integração entre sistemas.

Outro exemplo está no desenvolvimento orientado a objetos, onde contratos ajudam a garantir que subclasses respeitem os comportamentos definidos pelas superclasses.

Além disso, contratos podem ser utilizados em testes automatizados, funcionando como validações formais do comportamento esperado do sistema.

Em sistemas críticos, como aplicações bancárias ou sistemas médicos, essa abordagem se torna ainda mais importante, pois ajuda a reduzir falhas e aumentar a confiabilidade das operações.

Outro uso relevante está na documentação técnica. Contratos tornam a comunicação entre equipes mais clara, facilitando manutenção e evolução do software ao longo do tempo.

Mesmo em linguagens que não possuem suporte nativo para Design by Contract, muitos conceitos podem ser aplicados utilizando validações, testes automatizados e boas práticas de desenvolvimento.

VIII. CONCLUSÃO

O artigo de Bertrand Meyer apresenta uma abordagem extremamente relevante para a engenharia de software, propondo o uso de contratos explícitos como mecanismo para aumentar a confiabilidade e a qualidade dos sistemas.

Os conceitos de pré-condições, pós-condições e invariantes oferecem uma maneira organizada de definir responsabilidades entre componentes, reduzindo ambiguidades e melhorando a manutenção do software.

Além disso, a abordagem contribui para documentação, testes, reutilização e tratamento de erros, tornando os sistemas mais robustos e previsíveis.

Dessa forma, o Design by Contract se mostra uma técnica importante tanto do ponto de vista teórico quanto prático, permanecendo relevante mesmo diante das tecnologias modernas de desenvolvimento.

REFERÊNCIAS

- [1] B. Meyer, "Design by Contract," Interactive Software Engineering Inc.